Database and Storage Architecture Specification

PostgreSQL Schema, pgvector Indexing, Alembic Migrations, and Storage

1. Relational Schema & Entity Relationships

PostgreSQL serves as the single source of truth for all application and vector data. Tables are designed with strict foreign key constraints, cascading deletions, and compound indexes:

- **users:** id (UUID PK), email (UK index), hashed_password, full_name, role, is_active, created_at.
- **refresh_tokens:** id (UUID PK), user_id (FK), token (UK index), expires_at, is_revoked, created_at.
- **documents:** id (UUID PK), user_id (FK index), title, filename, file_path, file_size, mime_type, file_hash (index), status (index), page_count, chunk_count, metadata_json.
- **document_chunks:** id (UUID PK), document_id (FK index CASCADE), chunk_index, content (Text), page_number, token_count, embedding (Vector(384)), metadata_json.
- **conversations:** id (UUID PK), user_id (FK index), title, created_at, updated_at.
- **messages:** id (UUID PK), conversation_id (FK index CASCADE), role, content, citations_json (JSON), tokens_used, created_at.

2. Vector Indexing with pgvector

Document chunks store dense vector embeddings directly in PostgreSQL using pgvector's vector(384) data type. Similarity searches utilize the cosine distance operator (<=>), calculating:

```
SELECT chunks.id, chunks.content, chunks.page_number, (1 - (chunks.embedding <=> :query_vec)) AS
 similarity
FROM document_chunks chunks
JOIN documents docs ON docs.id = chunks.document_id
WHERE docs.user_id = :user_id AND docs.status = 'completed'
ORDER BY chunks.embedding <=> :query_vec ASC
LIMIT :top_k;
```

This guarantees that user data isolation is enforced at the database query level: a user cannot retrieve chunks from documents owned by another user under any circumstances.

3. Storage Layer & Path Traversal Guard

The storage layer prevents path traversal attacks by normalizing relative paths and validating that the resolved absolute path starts strictly within the designated storage root:

```
clean_rel = os.path.normpath(relative_path).lstrip('/\\')
full_path = os.path.abspath(os.path.join(self.base_dir, clean_rel))
if not full_path.startswith(self.base_dir):
```

```
    raise ValueError(f'Path traversal detected: {relative_path}')
```